Sun *et al. Journal of Engineering and Applied Science* (2026) 73:423
https://doi.org/10.1186/s44147-026-01285-z

Journal of Engineering and Applied Science

RESEARCH Open Access

# Certificateless universal designated-verifier signature proofs for carbon-footprint verification

Zhangcai Sun[1*], Jian Hu[2], Ou Yang[1], Jun Li[1] and Bo Wu[1]

*Correspondence:
Zhangcai Sun
1014258769@qq.com
[1]Puer Power Supply Bureau of Yunnan Power Grid Co., Ltd., Puer, Yunnan, China
[2]Digitalization Department, China Southern Power Grid Co., Ltd., Guangzhou, Guangdong, China

**Abstract**

Carbon-footprint audits require organizations to verify signed electricity and production evidence while limiting unnecessary disclosure. We study a certificateless signature-proof construction that combines a partial key issued by a key-generation center (KGC), an independent user secret, signature transformation and an interactive proof of the transformation factor. A canonical message binds the report and evidence digests to the signer identity, key epoch and accounting context; detailed records remain off chain. For validated nonidentity statements, we establish correctness, two-transcript signature extraction and honest-verifier transcript simulation. These properties are distinguished from the additional unforgeability requirements of the underlying signature. Reproducible analytical calculations describe communication size, group-operation counts, queueing and ledger-dependent workflow latency. Under explicitly selected delay inputs, an asynchronous online decision is calculated at 385.2 ms, compared with 2570.2 ms when commitment blocks the same path. These values are model outputs, not measured cryptographic or deployment performance. Source attestation, endpoint key protection, revocation and constrained-device verification are discussed as deployment measures. The result is a bounded design and analytical assessment; a complete underlying-signature security proof and implementation measurements remain necessary for deployment.

**Keywords** Certificateless cryptography, Universal designated-verifier signature proof, Carbon-footprint verification, Permissioned blockchain, Analytical performance model

## Introduction

Product carbon-footprint verification requires evidence from manufacturers, power-grid companies, certification bodies and authorized buyers. Electricity-intensive products depend particularly on metered consumption, electricity purchase records, renewable-energy certificates and production-batch information. ISO 14067 provides a framework for quantifying product carbon footprints, while the Carbon Border Adjustment Mechanism (CBAM) motivates the exchange of carbon information across jurisdictions [1, 2].

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Springer

These frameworks motivate the application; the protocol studied here neither computes emissions nor establishes regulatory compliance.

The evidence can reveal production patterns, supplier relationships and purchasing terms. A report holder may therefore wish to demonstrate possession of an authentic signed report in an authorized session without giving the recipient a transcript that reliably establishes the holder's participation to an outsider. This goal concerns the evidentiary value of a verification transcript. It does not prevent a recipient from copying report content, and it does not make false measurements truthful.

A permissioned ledger can record digests, issuer identities, key-status events and audit timestamps, while detailed evidence remains in controlled off-chain storage. A ledger anchor alone does not provide non-transferability: an ordinary public signature can still be forwarded and verified. An interactive signature-proof interface must therefore be accompanied by controls over the original signature, the report and the communication channel.

Identity-based universal designated-verifier signature proofs simplify key management but give the key-generation center (KGC) enough information to derive complete signing keys. Certificateless designs instead combine an authority-issued partial key with an independently generated user secret. We examine a construction, denoted UDVSPecls, that uses this split-key structure, randomized signature transformation and a proof of the transformation factor. The application is carbon-evidence verification across organizational trust boundaries.

The contribution is threefold. First, we specify the signed evidence object, session handling and deployment responsibilities, including source attestation and key revocation. Second, we establish algebraic correctness, two-transcript extraction and honest-verifier transcript simulation for a validated proof statement. The security of the exact underlying signature against public-key replacement and malicious-KGC attacks is an additional requirement, not a consequence of those algebraic properties. Third, we provide reproducible analytical calculations for communication size, operation counts, queueing and ledger-dependent latency. The calculations use explicitly selected scenario inputs and are not cryptographic benchmarks or production measurements.

The analysis separates cryptographic properties from operational effects so that the impact of a selected service time can be examined without assuming that one hardware platform represents every deployment. The comparison concerns the carbon-evidence workflow and does not establish performance superiority over recent signature-proof constructions.

## Related work

Carbon-footprint verification combines life-cycle assessment, regulatory compliance, and multi-party data governance. ISO 14067 provides the quantification vocabulary, while CBAM makes the reporting problem commercially urgent. For electricity-intensive products, the carbon evidence is deeply tied to power-system operation, because the same unit of production may have different carbon intensities under different time-of-use energy mixes, renewable certificate policies, or regional grid factors [1, 2].

Recent carbon-traceability systems often rely on distributed ledgers and off-chain storage. Ju et al. propose a blockchain-based traceability approach for enterprise and product carbon footprints, and Dutta et al. survey blockchain use across supply-chain

operations. These studies motivate the use of ledger anchors, but they also reveal the performance and privacy limits of storing and querying rich supply-chain evidence directly on chain [3, 4].

Permissioned ledgers such as Hyperledger Fabric are relevant because industrial carbon verification usually involves known organizations rather than anonymous miners. Fabric separates endorsement, ordering, validation, and state-database access, which allows membership control and enterprise governance. This architecture suits carbon verification. Endorsement and ordering add configuration-dependent delay, so their costs must be separated from cryptographic verification; seconds-level writes are assumptions of the analytical examples, not an inherent Fabric requirement [5].

Identity-based cryptography was introduced to remove conventional certificate management. Boneh and Franklin made pairing-based identity cryptography practical, and these ideas strongly influenced identity-based signatures and designated-verifier constructions. The price is escrow: the authority that generates private keys is also capable of impersonating users [6, 7].

Certificateless public-key cryptography was proposed to remove the escrow weakness while avoiding the full certificate burden of PKI. A KGC computes only a partial private key, while the user contributes an independent secret value. This split-key model separates identity registration from the user-controlled signing contribution. Preventing KGC-only forgery nevertheless requires security of the specific signature construction, not merely a split in key storage [8].

Designated-verifier signatures and UDVSP schemes support non-transferable verification. Steinfeld et al. formalized universal designated-verifier signatures, and Baek et al. introduced a signature-proof perspective in which the proof holder efficiently proves knowledge of a signature to a designated verifier. UDVSPecls follows this direction but adapts it to a certificateless trust model and power-carbon verification workflow [9, 10].

Recent studies have reduced designated-verification costs under different key models. Yang et al. construct pairing-free identity-based UDVSP systems from the ID-based SM2 signature and also consider an OR-proof/Fiat–Shamir variant [11]. Their identity-based key-generation model differs from a certificateless split-key model. Thanalakshmi et al. study a quantum-resistant UDVSP based on coding-theory assumptions [12]. That construction is code-based, not lattice-based, and does not make the present pairing-based protocol post-quantum secure.

Recent certificateless work includes designated-verifier sanitizable signatures. Zhan et al. combine controlled record modification with designated verification in mobile healthcare, while Sun et al. present a pairing-free certificateless sanitizable construction [13, 14]. These schemes permit authorized changes to signed records and should be distinguished from an interactive proof of possession of an unchanged signature. They are relevant design alternatives, but we compare their key models and functionality rather than claim a numerical speed advantage.

The application examined here combines off-chain carbon evidence, ledger anchors, and a certificateless proof interface. The distinction from the cited literature is the application and trust model under study; the available comparison does not establish that this is the first or most efficient combination of these components.

## Application scenario and requirements

The proposed consortium contains five functional roles: a KGC, a report signer, a report holder, an authorized verifier and a permissioned ledger. A grid company, manufacturer or certification agency may act as signer. The holder can be the producer or another authorized custodian. A procurement auditor, customs reviewer or power-market participant may act as verifier, subject to the applicable access policy.

The signer creates an evidence-bound report digest and a signature. The holder retains the original signature and performs a fresh interactive proof when an authorized verifier requests it. The verifier checks current key status, the signed object and any required existing ledger anchor. A new audit event may be anchored after the online proof, but the interface must distinguish proof completion from ledger commitment. Figure 1 summarizes the flow. No confidential report, original signature or interactive transcript needs to be published on chain.

Table 1 separates the desired properties from the mechanism or external control needed to obtain them. In particular, key splitting is an architectural prerequisite for escrow resistance, not a substitute for an unforgeability proof. Non-transferability is limited here to honest-verifier transcript simulation; authenticated session selection supplies access control.

## Power-carbon data model

Let $R$ be a confidential report and $E$ its evidence bundle. The public policy context $P$ identifies the accounting policy and its version. A stable record identifier rid distinguishes report records; batch identifies the production batch. These identifiers are not proof-session nonces. Repeated audits of one signed report use the same signed object but different proof sessions.

Let Enc be a versioned, injective encoding of a typed tuple. Each variable-length field has an explicit type and length; integers have a fixed byte order. A deployment must fix canonical encodings for structured reports and evidence manifests before hashing. The outer hash $H$ is a collision-resistant byte-string hash, distinct in purpose and domain from the hash-to-group functions used below. The signed message is

$$\begin{aligned} D_R &= H\left(\mathrm{Enc}(\mathrm{rid}, \mathrm{batch}, H(R), H(E), P)\right), \\ M &= \mathrm{Enc}(\texttt{JEAS- CFP- v1}, ID_A, \mathrm{epoch}, D_R). \end{aligned} \tag{1}$$

Every occurrence of $M$ in the signing and verification equations means precisely the second line of Eq. (1). An epoch-qualified identity can also be used for partial-key issuance. The authenticated key registry binds the identity and epoch to the active public key. A verifier must use the same identity, epoch, encoding version and digest when checking a signature or proof.

For a full-report audit, the verifier obtains the authorized $R$, $E$, $P$ and recomputes $D_R$ . For a digest-only session, the verifier demonstrates no independent knowledge of the hidden report contents: it checks the selected signed digest and its authorized anchor. The protocol does not provide selective disclosure of report attributes or a proof that an unseen field satisfies an accounting rule.

Electricity demand, supply attribution and grid factors can change across accounting periods, which makes the report interval and policy version material to carbon evidence [15]. Table 2 gives illustrative evidence classes, not sampled deployment data. A changed

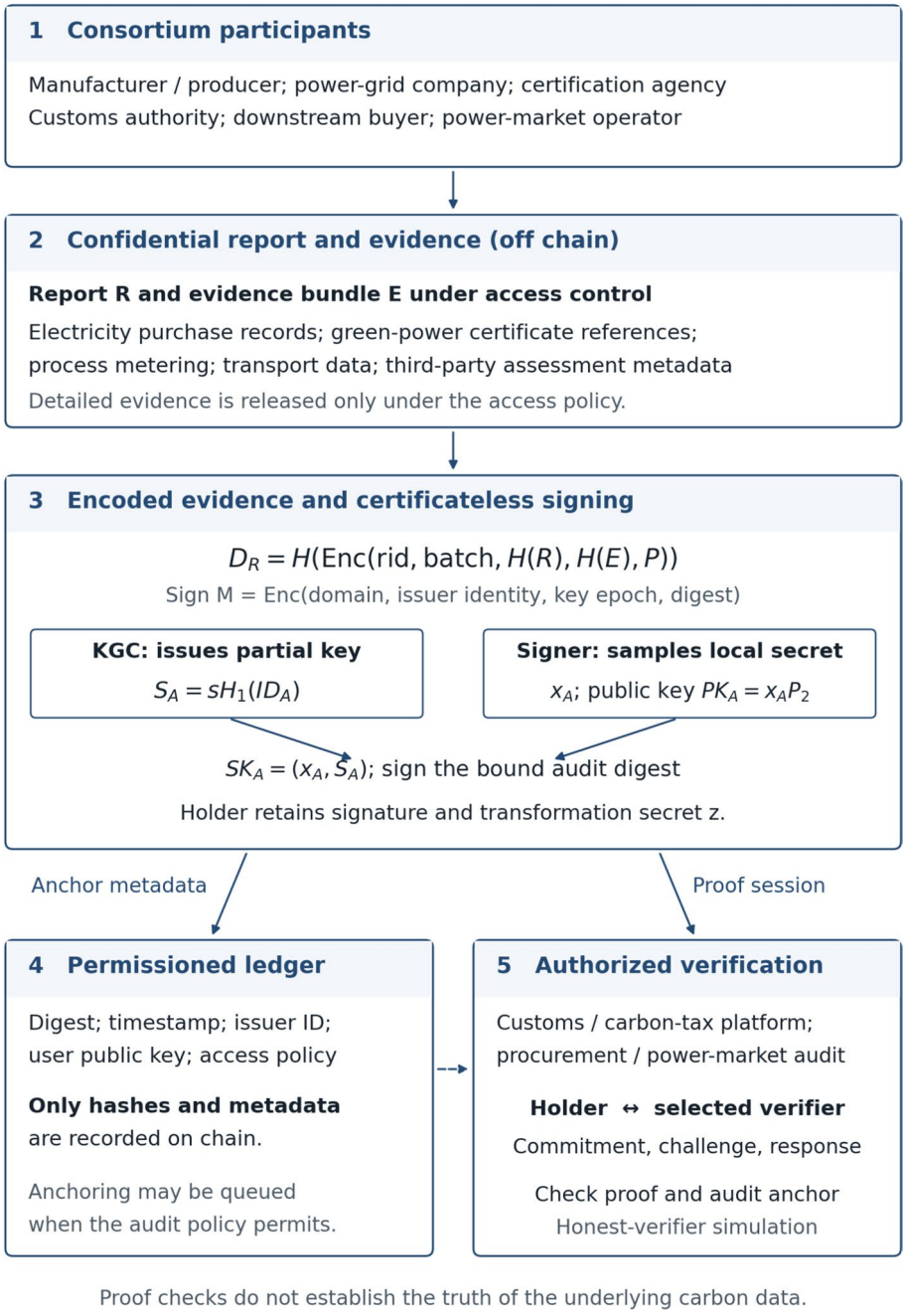



**Fig. 1** Carbon-evidence verification with separate proof and ledger paths

signed field is an integrity issue; reuse of an otherwise valid certificate or an unchanged old record additionally requires a certificate-use registry and freshness policy.

The ledger is an audit index. An entry binds the record identifier, issuer identity, key epoch, digest and commitment status. Access-controlled off-chain storage holds $R$ and $E$. Hashes of low-entropy values can still be guessed; data minimization and access control are needed even when only digests are published. Sessions use a separate fresh identifier sid and an expiry time. Neither a stable record identifier nor a ledger timestamp replaces a fresh interactive challenge.

**Table 1** Requirements and their supporting mechanisms

| Requirement | Mechanism | Boundary |
|---|---|---|
| Evidence integrity | Sign an encoded report and evidence digest | Does not establish measurement truth |
| Restricted transcript value | Interactive proof with a simulator | Honest-verifier scope; report copying remains possible |
| Escrow-resistant design | User secret independent of the KGC | Requires security of the exact signature |
| Traceable audit events | Ledger digests and metadata | Pending writes are not committed evidence |
| Responsive verification | Separate online proof and background anchoring | Device latency requires measurement |

**Table 2** Illustrative evidence classes and disclosure boundaries

| Evidence | Possible source | Sensitive content | Authorized check |
|---|---|---|---|
| Meter readings | Plant acquisition system | Interval load curve | Interval and device identity |
| Renewable certificate | Grid or certificate registry | Contract information | Identifier and permitted use |
| Production record | Manufacturing system | Supplier and process mix | Product-batch binding |
| Audit opinion | Certification body | Working papers | Signer authority and report digest |

## System and adversarial model

The KGC verifies registration and issues a partial private key. The user generates the independent local secret and publishes the associated public key through an authenticated registry. The report holder may know the original signature without knowing the signer's private key; possession of that signature is sufficient for legitimate transformation. The verifier authenticates the holder's session and checks the current identity–key binding before starting the proof.

Two adversarial classes motivate the design. A Type I adversary is external to the KGC: it does not know the master secret but may try to replace registered public keys. A Type II adversary knows the master secret but does not replace the target public key or obtain the target local secret. Public parameters are generated and validated as specified; maliciously selected trapdoored parameters are outside this Type II interpretation.

The formal claims made in Correctness and security analysis section concern a fixed, validated statement and its interactive proof. A full adaptive unforgeability theorem for the underlying signature must specify extraction and signing oracles, public-key replacement semantics, and freshness of the target identity, key and message. In particular, a challenger cannot simultaneously give an attacker a target partial key and allow that attacker to install a locally known complete signing key while still calling the resulting signature a forgery. We do not infer such a theorem merely from the split-key equation.

For interpreting signature extraction, a fresh target excludes prior access to a valid original signature or transformation for the same target tuple, and excludes live assistance from an honest holder. Otherwise an accepting session may be legitimate use of an already obtained signature. Full endpoint compromise, pre-signing measurement fraud, live relaying, malicious-verifier concurrency and denial of service are not covered by the algebraic analysis.

### Data origin before report signing

An optional data-origin layer can be placed before report signing. Each sensor or acquisition gateway supplies a measurement record containing its identity, measurement interval, sequence number and value digest, together with a hardware-backed attestation

that binds a fresh nonce and the digest to the relevant software measurement. An attestation verifier checks the endorsement chain, acceptable reference values and freshness before the report signer incorporates the attestation evidence into $E$. Because $D_R$ already includes $H(E)$, subsequent substitution of that evidence changes the signed report digest. This proposed flow follows the separation of attester, verifier and relying party in the RATS architecture [16].

A hardware security module (HSM) can protect a signing key, and an attested trusted execution environment (TEE), such as an appropriately configured Intel Software Guard Extensions (SGX) enclave, can help protect the acquisition and aggregation code. Neither mechanism establishes that a physical sensor was calibrated correctly or that its input was truthful. The trusted acquisition path, sensor commissioning, calibration records, reconciliation against independent meters and certificate-use registry remain necessary. In particular, enterprise–auditor collusion that produces false data before signing remains outside the signature-proof guarantee. This integration is an architectural proposal and is not an implemented component of the analytical evaluation.

#### Endpoint protection and key revocation

The local secret should be generated and used inside a suitable trusted execution boundary, with restricted signing authorization and protected randomness. An HSM or programmable enclave is useful only if it supports the required operations; a Trusted Platform Module (TPM) that seals key material does not by itself guarantee that a custom pairing-based signing operation executes inside the TPM. If unsealing exposes the key to host memory, host compromise remains relevant. Keeping the partial key and local secret in separate files is not protection against an attacker that controls the entire host. These controls reduce exposure but do not extend the stated model to full-key compromise [17].

For deployment, a consortium-maintained authenticated registry can record $(ID_A, H(PK_A), \text{epoch}, \text{status}, \text{validity})$. Before issuing a challenge, the verifier checks the current key status and rejects revoked, expired or stale entries according to the audit policy. After a reported compromise, operators revoke the affected key, terminate pending sessions, invalidate caches and register a newly generated local key; the partial key can also be reissued under an epoch-qualified identity. A revocation service must be available even when ledger anchoring is delayed. Historical signatures require a separate policy based on trustworthy pre-compromise timestamps; key rotation does not provide retroactive protection or forward security. These are proposed lifecycle controls, not a formally analyzed revocable variant.

## The UDVSPecls construction

Let $\mathbb{G}_1, \mathbb{G}_2$ be additive cyclic groups of prime order $q$, and $\mathbb{G}_T$ a multiplicative cyclic group of the same order. Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a non-degenerate asymmetric bilinear pairing. The parameters contain generators $P_1, P_2$, a nonidentity element $Q_1 \in \mathbb{G}_1$ and domain-separated hash-to-group functions $H_1, H_2$ for identities and messages. Hash-to-group outputs and received group elements are validated in the intended prime-order subgroups. Public keys and fixed generators must be nonidentity, and encodings must be canonical.

#### Setup and signing

The KGC samples $s \in \mathbb{Z}_q^*$, publishes $P_{\text{pub}} = sP_2$ and retains $s$. It issues $S_A = sH_1(ID_A)$ over an authenticated confidential channel. The user verifies

$$e(S_A, P_2) = e(H_1(ID_A), P_{\text{pub}}). \tag{2}$$

The user independently samples $x_A \in \mathbb{Z}_q^*$ and sets $PK_A = x_A P_2$, $SK_A = (x_A, S_A)$. On the encoded message $M$ from Eq. (1), signing samples fresh uniform $u \in \mathbb{Z}_q$ and computes

$$U = uP_2, \qquad V = S_A + x_A H_2(M) + uQ_1, \qquad \sigma = (U, V). \tag{3}$$

A public verifier accepts the original signature only when all domain and key-status checks pass and

$$e(V, P_2) = e(H_1(ID_A), P_{\text{pub}})e(H_2(M), PK_A)e(Q_1, U). \tag{4}$$

The displayed equation has four pairing terms when evaluated directly. The master secret alone does not supply $x_A$; the resulting intended resistance to KGC-only forgery still depends on the security of this exact signature construction.

#### Transformation and interactive verification

The holder first validates the original signature, then samples fresh $z \in \mathbb{Z}_q^*$ and forms $\widetilde{U} = zU$, $\widetilde{V} = zV$. The verifier receives the transformed pair and the agreed message tuple, and computes

$$\begin{aligned} T &= e(H_1(ID_A), P_{\text{pub}})e(H_2(M), PK_A), \\ W &= e(\widetilde{V}, P_2)e(Q_1, -\widetilde{U}). \end{aligned} \tag{5}$$

Both $T$ and $W$ must be nonidentity. A statement with $T = 1$ is outside the admitted proof domain and must be rejected even if the underlying public signature passes Eq. (4). This restriction is needed for the extraction statement; universal completeness for such excluded statements is not claimed.

The prover samples fresh $\alpha \in \mathbb{Z}_q$ and sends $a = T^\alpha$. After receiving $a$, the verifier independently samples uniform $c \in \mathbb{Z}_q$. The prover returns

$$t = \alpha + cz \pmod{q}, \qquad \text{and the verifier checks } T^t = aW^c. \tag{6}$$

Received scalars use canonical encodings in the stated range. The commitment belongs to $\mathbb{G}_T$; it need not be nonidentity because $\alpha = 0$ is permitted. By contrast, $W = 1$ must be rejected. Without that test an attacker could choose $\widetilde{U} = rP_2$, $\widetilde{V} = rQ_1$, obtain $W = 1$ and answer any challenge with $t = \alpha$ without a signature.

The session record binds sid, the verifier identity, $ID_A$, key epoch, $M$, the transformed signature, $a$, $c$, expiry and completion state. The channel authenticates both endpoints. A challenge is generated only after the statement and commitment are fixed, and the session is consumed atomically on completion. Reusing $\alpha$ with different challenges exposes $z$. This state handling prevents ordinary transcript reuse; it does not prevent an authorized participant from forwarding a live exchange.

## Correctness and security analysis

The following propositions concern the admitted prime-order statement domain of The UDVSPecls construction section. They distinguish verifiable algebraic properties from security requirements of the underlying signature.

#### Proposition 1 Correctness on admitted statements

For an honestly generated signature and nonzero transformation factor, bilinearity gives

$$W = e(zV, P_2)e(Q_1, -zU) = (e(V, P_2)e(Q_1, -U))^z = T^z. \tag{7}$$

If $T \neq 1$, then $W \neq 1$ because $T$ has prime order and $z \neq 0$. Substituting $t = \alpha + cz$ gives $T^t = T^\alpha (T^z)^c = aW^c$, so an honest session on an admitted statement is accepted.

#### Proposition 2 Two-transcript extraction

Fix the same validated statement $(T, W, \widetilde{U}, \widetilde{V})$ and commitment $a$. Suppose two transcripts (*a*, *c*, *t*) and (a, c', t') are accepted with $c \neq c'$ . Division of their verification equations gives

$$z = (t - t')(c - c')^{-1} \pmod{q}, \qquad T^z = W. \tag{8}$$

The inverse exists because $q$ is prime. Since $W \neq 1$, $z \neq 0$. Define $U^* = z^{-1}\widetilde{U}$ and $V^* = z^{-1}\widetilde{V}$. Raising the relation in Eq. (5) to $z^{-1}$ yields

$$e(V^*, P_2)e(Q_1, -U^*) = T, \tag{9}$$

which is equivalent to the original signature-verification equation. Thus two accepting transcripts yield a valid underlying signature algebraically. This statement is exact; it does not itself provide a probabilistic extractor for every adaptive adversary or simulate that adversary's oracle access.

#### Proposition 3 Honest-verifier transcript simulation

For any admitted statement with witness $z$, sample independent uniform $c, t \in \mathbb{Z}_q$ and set

$$a = T^t W^{-c}. \tag{10}$$

The generated transcript accepts. In an honest exchange, conditional on $c$, $t = \alpha + cz$ is uniform because $\alpha$ is uniform. Moreover $a = T^{t-cz}$ equals the simulated value. Hence the two transcript distributions are identical for a fixed true statement. An offline transcript alone cannot establish that the holder participated. Simulation does not let an attacker choose $a$ after seeing a fresh online challenge, and does not establish arbitrary malicious-verifier or concurrent security.

#### What the propositions establish

The propositions establish correctness, special soundness in the two-transcript sense and honest-verifier simulation for the proof relation. A conditional implication to fresh-target signature forgery additionally requires a valid rewinding reduction, a compatible adaptive oracle simulation and unforgeability of the exact signature in the relevant Type

**Table 3** Assurance scope of the construction and deployment controls

| Property | Basis | Scope |
|---|---|---|
| Proof correctness | Proposition 1 | Admitted nonidentity statements |
| Signature recovery | Proposition 2 | Two transcripts; same commitment |
| Transcript simulation | Proposition 3 | Honest-verifier distribution |
| Type I/II unforgeability | Underlying-signature theorem required | Not established by this analysis |
| Replay and revocation | Session state and current key status | Deployment controls |
| Measurement truth | Calibration and independent checks | Outside the proof guarantee |

**Table 4** Selected analytical inputs and their interpretation

| Input | Value | Interpretation |
|---|---|---|
| Nominal mean proof service | 329.2 ms | Illustrative input, not benchmark |
| Existing-anchor read | 19 ms | Fixed scenario delay |
| Network contribution | 23 or 37 ms | Fixed scenario delay |
| Ledger commitment delay | 2185 ms | Fixed scenario delay |
| Queue workers | 4, 8, 16, 32 | Parallel identical servers |
| Queue arrivals | 8, 16, 32, 64, 96/s | Open Poisson input rates |
| Sensitivity mean service | 200, 329.2, 450, 500 ms | Fixed$k = 8$, arrivals 16/s |

I or Type II model. These requirements are not replaced by zero false accepts in a finite test set. No complete Type I/II unforgeability theorem for the displayed signature is claimed here.

Designation means that the holder chooses an authenticated, authorized session. The equations do not contain a verifier-specific secret key. This is different from identity-exclusive public verification or a guarantee that a recipient cannot disclose data. The original publicly verifiable signature must therefore remain protected; releasing it would allow an outsider to verify it directly or perform its own transformation.

Table 3 records the assurance boundaries. The distinction matters operationally: a fresh challenge addresses replay, a registry addresses revocation and certificate reuse, and source attestation addresses provenance. None of these controls establishes that a physical measurement is accurate.

## Methods

The evaluation is analytical and reproducible from the accompanying Python program, `reproduce_analysis.py`. It contains no measured cryptographic timings, Fabric deployment results, CPU or memory measurements, empirical confidence intervals, or observed attack-rejection rates. Symbolic operation counts are derived from the displayed equations. Queueing and workflow values are calculated from declared inputs; no pseudorandom timing noise or synthetic repetitions are used.

### Input assumptions and workflow model

Let $s_0 = 0.3292$ s be an illustrative mean proof-service time. This value is a scenario parameter, not a measured latency or a claim about a particular curve, library or processor. Additional cases vary it between 0.2 and 0.5 s. The read, network and write values in Table 4 are also selected inputs, not properties of Hyperledger Fabric. They allow a controlled comparison of whether a ledger write blocks the online response.

For a request, write $d_r, d_n, d_w$ for read, network and blocking ledger-write delays. The mean online response model is

$$\overline{L}_{\text{online}} = s_0 + \overline{W}_q + d_r + d_n + d_w. \tag{11}$$

The isolated workflow comparison sets $\overline{W}_q = 0$ to separate architectural delay from load-dependent queueing. For an asynchronous decision, $d_w = 0$ on the online path. This does not remove the background work or predict its completion time. Additional attestation, encoding, key-status and hardware costs must be measured before deployment; they are not assigned invented numerical overheads here.

#### Queueing calculation

We use the standard $M/M/k$ model: Poisson arrivals of rate $\lambda$, independent exponential service times of mean $s_0$, $k$ identical servers, first-come first-served scheduling, and an unbounded waiting room with no abandonment [18]. This is an analytical approximation; it is not the same as constant per-request cryptographic execution time. Let $\mu = 1/s_0$, offered load $A = \lambda s_0$ and utilization $\rho = A/k$. A steady state requires $\rho < 1$.

$$C(k, A) = \frac{\dfrac{A^k}{k!(1-\rho)}}{\displaystyle\sum_{n=0}^{k-1} \frac{A^n}{n!} + \frac{A^k}{k!(1-\rho)}}. \tag{12}$$

The waiting and response metrics are calculated as

$$\overline{W}_q = \frac{C(k, A)}{k\mu - \lambda}, \quad \overline{R} = s_0 + \overline{W}_q, \quad \overline{N} = \lambda \overline{R}. \tag{13}$$

Here $\overline{N}$ is the mean number in service plus queue. It is an output of an open-arrival model, not a configured concurrency limit. The queue tables contain neither end-to-end latency percentiles nor deployment throughput measurements. The calculation script also checks its one-server limit and the occupancy identity in Eq. (13), and rejects unstable cases instead of assigning a finite steady-state delay.

#### Communication and operation counting

Let $\ell_1, \ell_2, \ell_T, \ell_q$ be the encoded byte lengths of a $\mathbb{G}_1$ element, a $\mathbb{G}_2$ element, a $\mathbb{G}_T$ element and a scalar. The original and transformed signatures have the same two group-element types. Their size and the proof-transcript size are

$$\begin{aligned} C_\sigma = C_{\tilde{\sigma}} &= \ell_1 + \ell_2, \\ C_{\text{transcript}} &= \ell_T + 2\ell_q, \\ C_{\text{core session}} &= \ell_1 + \ell_2 + \ell_T + 2\ell_q. \end{aligned} \tag{14}$$

The last line assumes one transmission of the transformed signature followed by commitment, challenge and response. It excludes the message tuple, reports, identifiers, authentication, transport headers and ledger messages. A numerical byte count requires a chosen curve and serialization; no fixed 280-byte result is asserted.

## Results

All numerical values in this section are calculated from Methods section; the companion CSV and JSON files retain unrounded values. Table 5 illustrates the blocking cost of commitment under the selected inputs. The local path totals 352.2 ms, and an

**Table 5** Calculated workflow totals under selected delay inputs

| Workflow | Service (ms) | Read (ms) | Network (ms) | Blocking write (ms) | Total (ms) |
|---|---|---|---|---|---|
| Local proof | 329.2 | 0 | 23 | 0 | 352.2 |
| Existing-anchor read | 329.2 | 19 | 37 | 0 | 385.2 |
| Asynchronous online decision | 329.2 | 19 | 37 | 0 | 385.2 |
| Synchronous committed decision | 329.2 | 19 | 37 | 2185 | 2570.2 |

**Table 6** Calculated queueing metrics with nominal mean service time

| Workers | Arrivals/s | $\rho$ | $C(k, A)$ | Mean wait (ms) | Mean response (ms) |
|---|---|---|---|---|---|
| 4 | 8 | 0.6584 | 0.3664 | 88.280 | 417.480 |
| 8 | 16 | 0.6584 | 0.2095 | 25.234 | 354.434 |
| 16 | 32 | 0.6584 | 0.0843 | 5.077 | 334.277 |
| 32 | 64 | 0.6584 | 0.0179 | 0.538 | 329.738 |
| 32 | 96 | 0.9876 | 0.9181 | 761.718 | 1090.918 |

**Table 7** Sensitivity to selected mean service time

| Service (ms) | $\rho$ | Mean wait (ms) | Mean response (ms) | Stable |
|---|---|---|---|---|
| 200.0 | 0.4000 | 0.771 | 200.771 | Yes |
| 329.2 | 0.6584 | 25.234 | 354.434 | Yes |
| 450.0 | 0.9000 | 394.612 | 844.612 | Yes |
| 500.0 | 1.0000 | – | – | No |

A dash denotes no finite steady-state value because $\rho \geq 1$

existing-anchor read totals 385.2 ms. Adding a blocking 2185 ms write to that same read path gives 2570.2 ms. An asynchronous online response remains 385.2 ms in this isolated example, but the newly submitted audit record is still pending. These are arithmetic scenario totals, not observed protocol performance.

Table 6 reports steady-state queueing calculations. The first four rows have the same utilization, 0.6584, because both arrival rate and worker count scale together. Their mean waiting times decrease as capacity is pooled across more servers. At 32 workers and 96 arrivals/s, utilization is 0.9876 and mean waiting rises to approximately 761.7 ms. These outputs follow one specified model; no empirical CPU-utilization percentage is inferred from them.

The distinction between mean service time and mean response time is essential. For example, the 32-worker, 96-arrivals/s case has a mean response of approximately 1090.9 ms before external network and ledger costs. A service may therefore exceed a one-second response target even if its mean proof work is below one second. The model supplies no guarantee for a finite sample or a bursty workload.

Table 7 varies mean proof service time at eight workers and 16 arrivals/s. At 500 ms the offered load equals capacity, so there is no finite steady-state mean in this model. The numerical increase at lower stable loads illustrates why a device-specific service measurement matters; a time reported on one host cannot be transferred to a constrained meter.

Table 8 derives direct group-operation counts from the equations, with keys and partial keys already available. Signing requires two scalar multiplications in $\mathbb{G}_1$ and one in $\mathbb{G}_2$. Public verification requires four pairing evaluations. On the verifier side, interactive verification also requires four pairing evaluations to obtain $T$, $W$, followed by two target-group exponentiations. The prover separately computes $T$, forms its commitment and

**Table 8** Direct group-operation counts without cached pairing terms

| Operation | $\mathbb{G}_1$ scalar mult. | $\mathbb{G}_2$ scalar mult. | Pairings | $\mathbb{G}_T$ exponentiations |
|---|---|---|---|---|
| Sign | 2 | 1 | 0 | 0 |
| Public Verify | 0 | 0 | 4 | 0 |
| Transform | 1 | 1 | 0 | 0 |
| IVerify verifier | 0 | 0 | 4 | 2 |
| IVerify prover | 0 | 0 | 2 | 1 |

**Table 9** Required regression cases for a future implementation

| Case | Required check | Expected handling |
|---|---|---|
| Changed report or evidence | Recompute the encoded message and digest | Reject mismatched signed object |
| Malformed group encoding | Canonical decoding and subgroup membership | Reject before proof arithmetic |
| Zero-witness construction | Require $T \neq 1$ and $W \neq 1$ | Reject degenerate statement |
| Reused session response | Fresh challenge and single-use session state | Reject stale or consumed session |
| Revoked public key | Authenticated current key status | Reject before challenge issuance |
| Repeated certificate use | Certificate registry and audit policy | Apply permitted-use rule |

returns a scalar response. Counts do not include validation, hashing, additions, group multiplication, key-status access or message handling, and are not a wall-clock cost comparison.

No attack acceptance rate is reported. Table 9 instead states implementation obligations and their expected checks. It is a regression-test specification, not a record of executed cryptographic tests. This preserves the distinction between a mathematical counterexample, an implemented check and a measured test campaign.

## Discussion

The analysis identifies two different design questions: whether the signed statement and proof have the required security properties, and whether a deployment can complete the relevant workflow within its latency budget. Faster queueing or ledger handling cannot repair an insecure signature; conversely, a valid signature proof cannot remove ledger commitment delay or correct false sensor input.

The functional contribution is the explicit connection between a carbon-evidence object, a split-key signing interface, an interactive proof and operational controls. The analytical calculations make performance assumptions inspectable. They do not establish a new implementation speed record or a deployment result. A compact proof transcript is only one part of session traffic: authentication, report transfer and ledger messages have separate costs.

### Computation on constrained devices

An implementation can consider caching validated identity-dependent pairing terms and using pairing-product implementations. Cached values must be indexed by identity, public key, parameter set and key epoch, and invalidated on revocation or rollover. The direct counts in Table 8 make the uncached baseline explicit; implementation-specific precomputation must be reported separately. Pairing-free identity-based UDVSP and

related certificateless designated-verifier constructions provide relevant alternatives, but their functionality and key assumptions differ [11, 14].

Independent proof sessions can run concurrently, and audit anchors may be submitted in application batches. Neither constitutes a proved cryptographic batch-verification scheme. Any aggregation proposal needs its own soundness argument and invalid-member handling; multiplying unweighted equations is not sufficient because erroneous terms can cancel.

An authenticated gateway can host verification for a constrained client, but that client then relies on the gateway's decision. The signer secret and holder transformation factor must not be delegated to an untrusted gateway. A pairing-free elliptic-curve construction changes the proof relation and requires a new certificateless security analysis. A lattice-based alternative additionally requires a post-quantum design and an assessment of key, proof and communication sizes. Ordinary elliptic-curve cryptography is not post-quantum, and replacing only the signature does not make a discrete-logarithm proof post-quantum secure. These are design directions, not measured optimizations.

#### Asynchronous audit anchoring

When audit policy permits, the online path authenticates the session, checks the evidence digest and current key status, reads any required existing anchor and completes the proof. It returns the proof decision together with an explicit "anchoring pending" status. A durable queue then submits a minimal audit event for endorsement, ordering, validation and commitment. The event is marked anchored only after valid commitment, with an idempotency identifier and retry policy to prevent duplicate records. Endorsement and Raft ordering are distinct stages [19].

The analytical workflow comparison isolates the selected 2185 ms blocking write delay from the online response. It does not make background completion instantaneous. Queue persistence, backpressure, retry limits, failed commitments and recovery after a worker crash require implementation testing. Revocation checks must remain available when the ledger is delayed. A regulatory or commercial decision that requires a committed new record must continue to wait for that commitment.

#### Deployment boundaries and remaining research

The KGC, signer, holder, verifier and ledger operator should be treated as separate trust domains. Access control, minimal disclosure and explicit session authentication support this separation [20]. Proof workers may be stateless between requests only if active sessions retain authenticated, single-use state elsewhere. A report recipient can still disclose received content; transcript simulation does not provide data-use enforcement.

Hardware attestation identifies a software and device state under an attestation trust model. It cannot replace calibration, independent meter reconciliation or certificate governance. Similarly, rotating a compromised signing key does not retroactively protect earlier exposures. Historical audit acceptance requires a clear timestamp and revocation policy.

The principal theoretical limitation is the absence of a complete unforgeability reduction for the exact underlying signature in a fully specified Type I/II model. The propositions prove narrower properties and should be used within that scope. The principal empirical limitation is that no pairing implementation, embedded device or Fabric

network is benchmarked here. A subsequent evaluation must specify the security level, curve, hash-to-group suite, encodings, hardware, software versions and timing boundaries; publish test logs; and implement the input checks and negative cases described above. Until then the numerical results are model outputs only.

## Conclusions

This study presents a certificateless signature-proof construction and deployment workflow for controlled verification of signed carbon evidence. A canonical signed message binds the report digest, evidence bundle, accounting context, signer identity and key epoch. The ledger stores audit metadata, while detailed reports and original signatures remain under access control.

For admitted nonidentity statements, the analysis establishes correctness, two-transcript signature extraction and honest-verifier transcript simulation. These results do not establish full adaptive Type I/II security of the underlying signature or the truth of the carbon measurements. The zero-witness exclusion and session checks are explicit parts of the specification.

The reproducible evaluation reports symbolic communication and operation counts and analytical workflow and queueing calculations. Under one declared example, keeping ledger commitment outside the online path changes the calculated response from 2570.2 to 385.2 ms; neither value is a measured protocol latency. The queueing analysis also shows why response time must be distinguished from proof-service time near saturation. Source attestation, key revocation, constrained-device computation and asynchronous anchoring are specified as deployment measures. Complete underlying-signature security and implementation measurements remain necessary before treating the construction as deployment-ready.

**Abbreviations**

| | |
|---|---|
| CBAM | Carbon Border Adjustment Mechanism |
| CPU | Central processing unit |
| CSV | Comma-separated values |
| ECC | Elliptic-curve cryptography |
| HSM | Hardware security module |
| ID | Identifier |
| ISO | International Organization for Standardization |
| IVerify | Interactive verification |
| JSON | JavaScript Object Notation |
| KGC | Key-generation center |
| NIST | National Institute of Standards and Technology |
| PKI | Public key infrastructure |
| RATS | Remote ATtestation procedureS |
| REC | Renewable energy certificate |
| SGX | Software Guard Extensions |
| SM2 | Chinese elliptic-curve public-key cryptographic algorithm |
| SP | Special Publication |
| TEE | Trusted execution environment |
| TPM | Trusted Platform Module |
| UDVSP | Universal designated-verifier signature proof |
| UDVSPecls | Certificateless construction studied in this paper |

**Acknowledgements**
Not applicable.

**Authors' contributions**
Conceptualization, Z.S. and J.H.; methodology, Z.S.;software, O.Y.; validation, J.H., J.L. and B.W.; formal analysis, Z.S.; investigation,O.Y.; resources, J.H.; data curation, J.L.; writing-original draft preparation, Z.S.;writing-review and editing, J.H., O.Y. and J.L.; visualization, B.W.; supervision,J.H.; project administration, J.H.; funding acquisition, J.H. All authors have read andagreed to the published version of the manuscript.

**Funding**
The authors received no financial support for the research, authorship, and/or publication of this article.

**Data availability**
The datasets used and/or analyzed duringthe current study are available from the corresponding author on reasonable request.

## Declarations

**Competing interests**
The authors declare that they have no competing interests.

Received: 25 August 2026 / Accepted: 17 October 2026
Published online: 21 October 2026

**References**

1. International Organization for Standardization (2018) ISO 14067:2018 Greenhouse gases – Carbon footprint of products – Requirements and guidelines for quantification. ISO, Geneva
2. European Parliament and Council of the European Union (2023) Regulation (EU) 2023/956 establishing a carbon border adjustment mechanism. Off J Eur Union L 130:52–104
3. Ju C, Shen Z, Bao F, Weng P, Xu Y, Xu C (2022) A novel credible carbon footprint traceability system for low carbon economy using blockchain technology. Int J Environ Res Public Health 19(16):10316. https://doi.org/10.3390/ijerph191610316
4. Dutta P, Choi TM, Somani S, Butala R (2020) Blockchain technology in supply chain operations: Applications, challenges and research opportunities. Transp Res E Logist Transp Rev 142:102067. https://doi.org/10.1016/j.tre.2020.102067
5. Androulaki E, Barger A, Bortnikov V, Cachin C, Christidis K, De Caro A, Enyeart D, Ferris C, Laventman G, Manevich Y, Muralidharan S, Murthy C, Nguyen B, Sethi M, Singh G, Smith K, Sorniotti A, Stathakopoulou C, Vukolić M, Weed Cocco S, Yellick J (2018) Hyperledger Fabric: A distributed operating system for permissioned blockchains. In: Proceedings of EuroSys 2018, article 30. ACM, New York. https://doi.org/10.1145/3190508.3190538
6. Shamir A (1985) Identity-based cryptosystems and signature schemes. In: Advances in Cryptology – CRYPTO '84. LNCS 196. Springer, Berlin, pp 47–53. https://doi.org/10.1007/3-540-39568-7_5
7. Boneh D, Franklin M (2001) Identity-based encryption from the Weil pairing. In: Advances in Cryptology – CRYPTO 2001. LNCS 2139. Springer, Berlin, pp 213–229. https://doi.org/10.1007/3-540-44647-8_13
8. Al-Riyami SS, Paterson KG (2003) Certificateless public key cryptography. In: Advances in Cryptology – ASIACRYPT 2003. LNCS 2894. Springer, Berlin, pp 452–473. https://doi.org/10.1007/978-3-540-40061-5_29
9. Steinfeld R, Bull L, Wang H, Pieprzyk J (2003) Universal designated-verifier signatures. In: Advances in Cryptology – ASIACRYPT 2003. LNCS 2894. Springer, Berlin, pp 523–542. https://doi.org/10.1007/978-3-540-40061-5_33
10. Baek J, Safavi-Naini R, Susilo W (2005) Universal designated verifier signature proof (or how to efficiently prove knowledge of a signature). In: Advances in Cryptology – ASIACRYPT 2005. LNCS 3788. Springer, Berlin, pp 644–661. https://doi.org/10.1007/11593447_35
11. Yang Y, Zhou X, Su B, Wu W (2025) Efficient identity-based universal designated verifier signature proof systems. Mathematics 13(5):743. https://doi.org/10.3390/math13050743
12. Thanalakshmi P, Anbazhagan N, Joshi GP, Yang E (2023) A quantum resistant universal designated verifier signature proof. AIMS Math 8(8):18234–18250. https://doi.org/10.3934/math.2023927
13. Zhan Y, Yang Y, Yi B, He R, Shi R, Zheng X (2024) A certificateless designated verifier sanitizable signature in e-health intelligent mobile communication system. Comput Commun 228:107935. https://doi.org/10.1016/j.comcom.2024.107935
14. Sun Q, Lu Y, Sun Y, Li J (2025) Certificateless sanitizable signature with designated verifier. IEEE Trans Emerg Top Comput 13(3):1019–1029. https://doi.org/10.1109/TETC.2025.3562050
15. International Energy Agency (2024) Electricity 2024: Analysis and forecast to 2026. IEA, Paris. https://www.iea.org/reports/electricity-2024. Accessed 9 Oct 2026
16. Birkholz H, Thaler D, Richardson M, Smith N, Pan W (2023) Remote ATtestation procedureS (RATS) architecture. RFC 9334. https://doi.org/10.17487/RFC9334
17. Barker E (2020) Recommendation for key management: Part 1 – General. NIST SP 800-57 Part 1 Rev. 5. https://doi.org/10.6028/NIST.SP.800-57pt1r5
18. Bertsekas DP, Gallager RG (1992) Data networks, Chapter 3, 2nd edn. Prentice Hall, Englewood Cliffs
19. Hyperledger Fabric (2026) Transaction flow. Hyperledger Fabric documentation. https://hyperledger-fabric.readthedocs.io/en/latest/txflow.html. Accessed 9 Oct 2026
20. Rose S, Borchert O, Mitchell S, Connelly S (2020) Zero trust architecture. NIST SP 800-207. https://doi.org/10.6028/NIST.SP.800-207

## Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.